

Date:
Student Name:
Student ID:

AI Workshop #10

Artificial Neural Networks

10.1. Introduction

In the previous course Introduction to AI, we have introduced and discussed **Artificial Neural Networks (ANN)**. In this workshop, we are going to learn all the basic Python implementation of ANN.

We will start with an introduction to neural networks and the installation of the relevant library.

We will then discuss **perceptrons** and how to build a classifier based on them.

After that, we'll go deeper and learn about **single-layer neural networks** and **multi-layer neural networks**.

Later, we will see how to use neural networks to build a **vector quantizer**.

We will analyze sequential data using recurrent neural networks, and finally we will use neural networks to build an **optical character recognition engine**.

By the end of this workshop, you will have a better understanding of these topics:

- An introduction to neural networks
- Building a Perceptron-based classifier
- Constructing a single-layer neural network
- Constructing a multilayer neural network
- Building a vector quantizer
- Analyzing sequential data using recurrent neural networks
- Visualizing characters in an Optical Character Recognition (OCR) database
- Building an Optical Character Recognition (OCR) engine

Let's begin with a revisit to ANNs.

10.2 Revisit to Artificial Neural Networks (ANN)

One of the fundamental premises of AI is to build systems that can perform tasks that would normally require human intelligence.

The human brain is amazing at learning new concepts. Why not use the model of the human brain to build a system?

A neural network is a model designed to loosely simulate the learning process of the human brain.

Neural networks are designed such that they can identify the underlying patterns in data and learn from them.

They can be used for various tasks such as classification, regression, and segmentation.

One drawback of neural networks is that we need to convert any given data into a numerical format before feeding it into the neural network.

For example, we deal with many different types of data including visual, textual, and time series. We need to figure out how to represent problems in a way that can be understood by neural networks.

To understand this process, let's first consider how to build a neural network, and then how to train one.

The following figure shows neural network in our brain.



10.3 Building a Neural Network

Some components of the human learning process are hierarchical.

We have various parts in our brain's neural network and each stage corresponds to a different granularity.

Some parts learn simple things and some parts learn more complex things. Let's consider the example of visually recognizing an object.

When we look at a box, the first part in our brain might identify simple things like corners and edges.

The next part identifies the generic shape and the part after that identifies what kind of object it is.

This process might differ for different brain functions, but you get the idea.

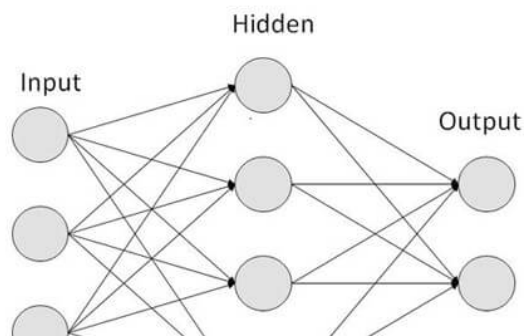
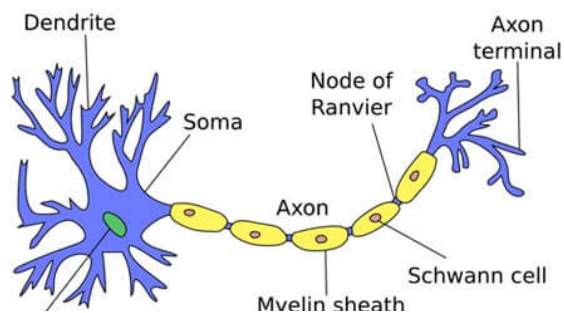
Using this hierarchy, the human brain separates the tasks and identifies the given object.

To simulate the learning process of the human brain, a neural network is built using layers of neurons.

These neurons are inspired by the biological neurons we discussed in the previous paragraph.

Each layer in a neural network is a set of independent neurons. Each neuron in a layer is connected to neurons in the adjacent layer.

The following figure shows the biological vs artificial neural networks.



10.4 Training a neural network

If we are dealing with N -dimensional input data, then the input layer will consist of N neurons.

If we have M distinct classes in our training data, then the output layer will consist of M neurons.

The layers between the input and output layers are called hidden layers.

A simple neural network will consist of a couple of layers and a deep neural network will consist of many layers.

So how can a neural network be used to classify data?

The first step is to collect the appropriate training data and label it.

Each neuron acts as a simple function and the neural network trains itself until the error goes below a certain threshold.

The error is the difference between the predicted output and the actual output.

Based on how big the error is, the neural network adjusts itself and retrain until it gets closer to the solution.

Enough about thinking of neural networks in the abstract.

Time to get our hands dirty and learn by doing, just like we have been doing throughout this book.

We will use a library called **NeuroLab** in this workshop.

NeuroLab is a library that implements basic neural network algorithms.

It has a variety of parameters that enables its configuration.

Its interface is like the **Neural Network Toolbox (NNT)** package in MATLAB.

The library is based on the NumPy package. You can find more about it at:

<https://pythonhosted.org/neurolab> (<https://pythonhosted.org/neurolab>)

You can install it by running the following command on your Terminal:

pip install neurolab

Once you have installed it, you can proceed to the next section where we will build a Perceptron-based classifier.

10.5 Building a Perceptron-based classifier

Neurons, dendrites, and axons comprise the building blocks of the brain.

Similarly, perceptrons are the most basic structure in a neural network.

The evolution of the neural networks has gone through many changes.

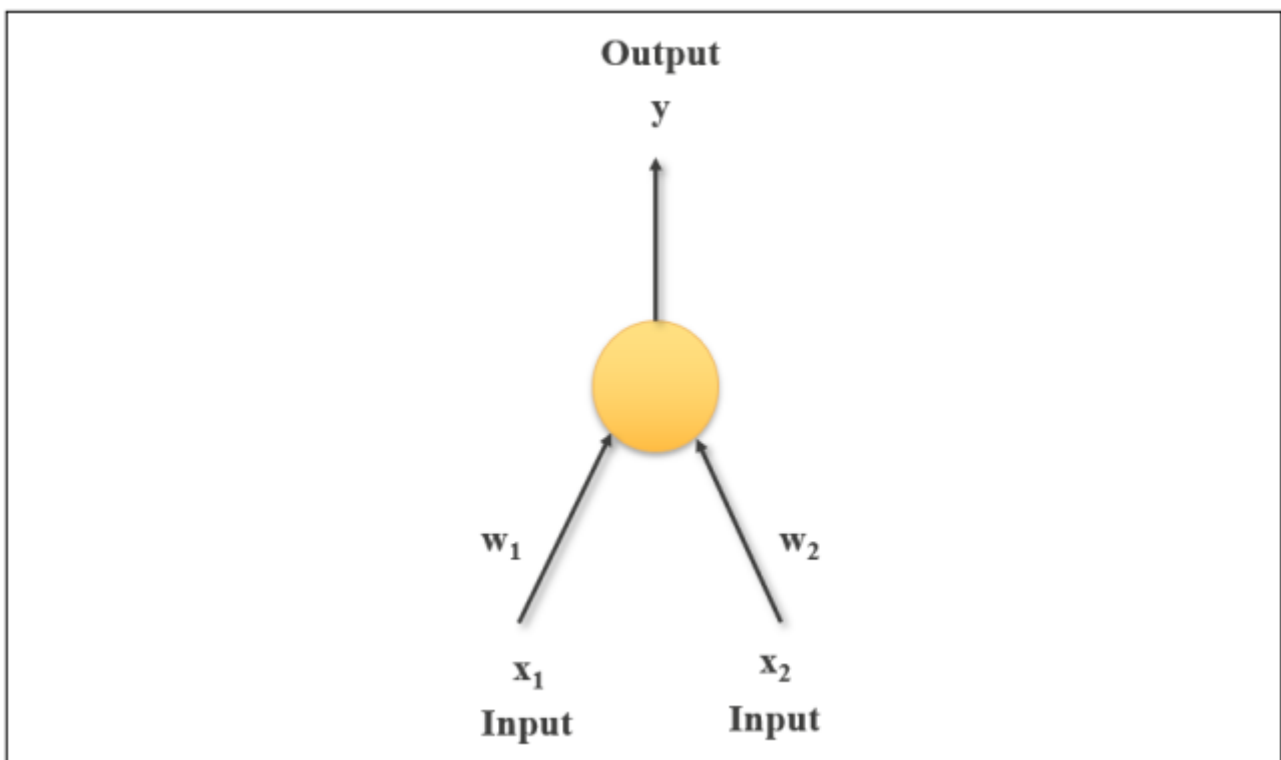
Their development is grounded on the neurological work performed by Santiago Ramon y Cajal and Sir Charles Scott Sherrington.

Ramon y Cajal was a pioneer in the exploration of the structure of nervous tissue and demonstrated that:

- Neurons can communicate with each other
- Neurons are physically separate from other neurons

Leveraging the research of Ramon y Cajal and Sherrington, Warren McCulloch and Walter Pitts in their 1943 paper, *A Logical Calculus of Ideas Immanent in Nervous Activity*, described an architecture borrowing from the structure of neurons having a binary threshold activation function analogous to first order logic sentences.

The following is a basic representation of McCulloch and Pitts neuron, otherwise called a Perceptron:



So, a Perceptron is a fundamental building block in many neural networks.

It takes inputs, performs computation on them, and then produces an output.

It uses a simple linear function to make the decision.

Let's say we are dealing with an N-dimension input data point.

A Perceptron computes the weighted summation of those N numbers and it then adds a constant to produce the output.

The constant is called the bias of the neuron. It is worth noting that these simple Perceptrons can be used to design complex deep neural networks.

In this workshop, we will see how such a basic structure can be used to perform machine learning.

Many neural networks, at their core, no matter how complicated they are, leverage the simple concept of the Perceptron. And that's why it's important to get a thorough understanding of this topic.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

2. Load the input data from the text file data_perceptron.txt provided to you. Each line contains space-separated numbers, where the first two numbers are the features and the last number is the label:

```
# Load input data
text = np.loadtxt('data_perceptron.txt')
```

3. Separate the text into data points and labels:

```
# Separate datapoints and labels
data = text[:, :2]
labels = text[:, 2].reshape((text.shape[0], 1))
```

4. Plot the data points:

```
# Plot input data
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Input data')
```

5. Define the maximum and minimum values that each dimension can take:

```
# Define minimum and maximum values for each dimension
dim1_min, dim1_max, dim2_min, dim2_max = 0, 1, 0, 1
```

6. Since the data is separated into two classes, we just need one bit to represent the output. So, the output layer will contain a single neuron.

```
# Number of neurons in the output layer
num_output = labels.shape[1]
```

7. We have a dataset where the data points are two-dimensional. Let's define a Perceptron with two input neurons, where we assign one neuron for each dimension.

```
# Define a perceptron with 2 input neurons (because we
# have 2 dimensions in the input data)
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
perceptron = nl.net.newp([dim1, dim2], num_output)
```

8. Train the perceptron with the training data:

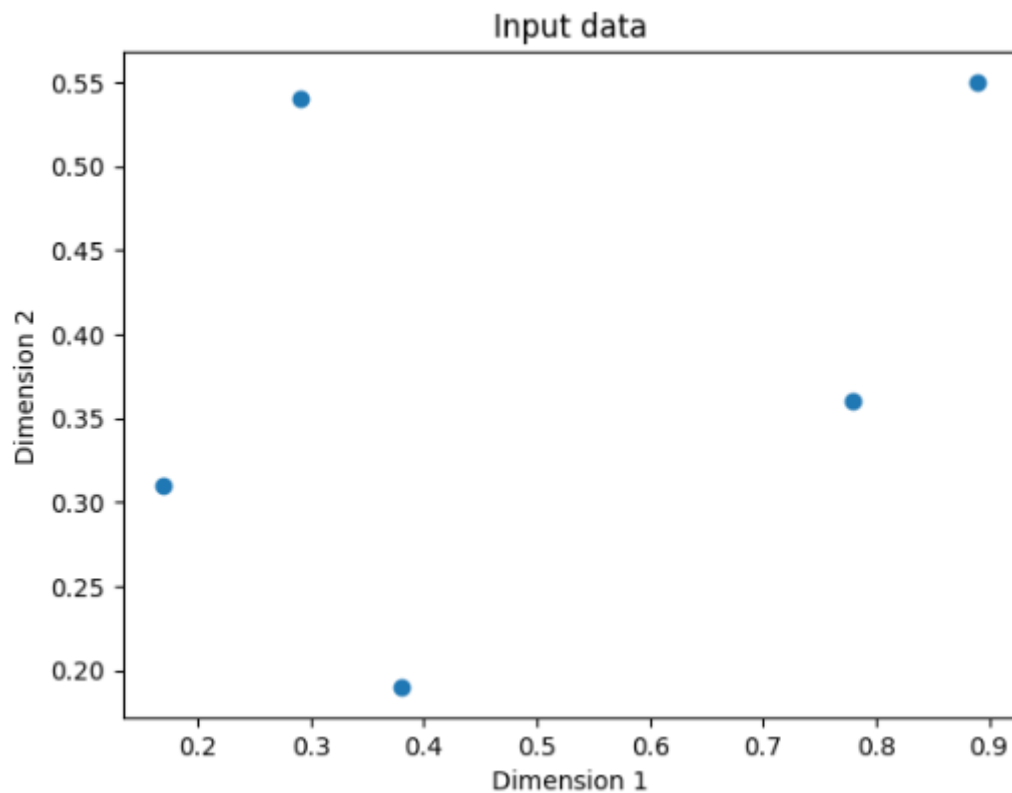
```
# Train the perceptron using the data
error_progress = perceptron.train(data, labels, epochs=100, show=20, lr=0.03)
```

9. Plot the training progress using the error metric:

```
# Plot the training progress
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Training error')
plt.title('Training error progress')
plt.grid()

plt.show()
```

10. Run the program, you will see 2 figures. The first one shows the input data points. The second shows the training progress using the error metric.





Workshop 10.1 : Perceptron Classifier

1. Combine all the above code and named it as **perceptron_classifier.py**.
2. If everything is okay, you should see the above 2 figures.
3. Save and submit the source code and these figures.
4. What is your observation from the second figure? Why the training error increase and then drop to zero after 4 epochs.
5. Try the experiment by using different learning rates, plot their training error and compare them. What is your findings?

10.6 Constructing a Single-layer Neural Network

Having a model that has a few perceptrons is a good start and it allowed us to get a fundamental understanding of this exciting concept, but to really solve problems, such a simple model will not be sufficient.

The human brain has approximately 85 billion neurons.

We won't be building a neural network that has that many nodes, but this number gives you an idea of what it takes to solve complicated problems.

Before we build a model with billions of nodes, let's take the next step to building a network with a single-layer.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

2. We will use the input data from the file `data_simple_nn.txt` provided to you. Each line in this file contains four numbers. The first two numbers form the datapoint and the last two numbers are the labels. Why do we need to assign two numbers for labels? Because we have four distinct classes in our dataset, so we need two bits to represent them. Let's go ahead and load the data:

```
# Load input data
text = np.loadtxt('data_simple_nn.txt')
```

3. Separate the data into data points and labels:

```
# Separate it into datapoints and labels
data = text[:, 0:2]
labels = text[:, 2:]
```

4. Plot the input data:

```
# Plot input data
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Input data')
```

5. Extract the minimum and maximum values for each dimension (we don't need to hardcode it like we did in the previous section):

```
# Minimum and maximum values for each dimension
dim1_min, dim1_max = data[:,0].min(), data[:,0].max()
dim2_min, dim2_max = data[:,1].min(), data[:,1].max()
```

6. Define the number of neurons in the output layer:

```
# Define the number of neurons in the output layer
num_output = labels.shape[1]
```

7. Define a single-layer neural network using the above parameters:

```
# Define a single-layer neural network
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
```

```
nn = nl.net.newp([dim1, dim2], num_output)
```

8. Train the neural network using training data:

```
# Train the neural network
error_progress = nn.train(data, labels, epochs=100, show=20, lr=0.03)
```

9. Plot the training progress:

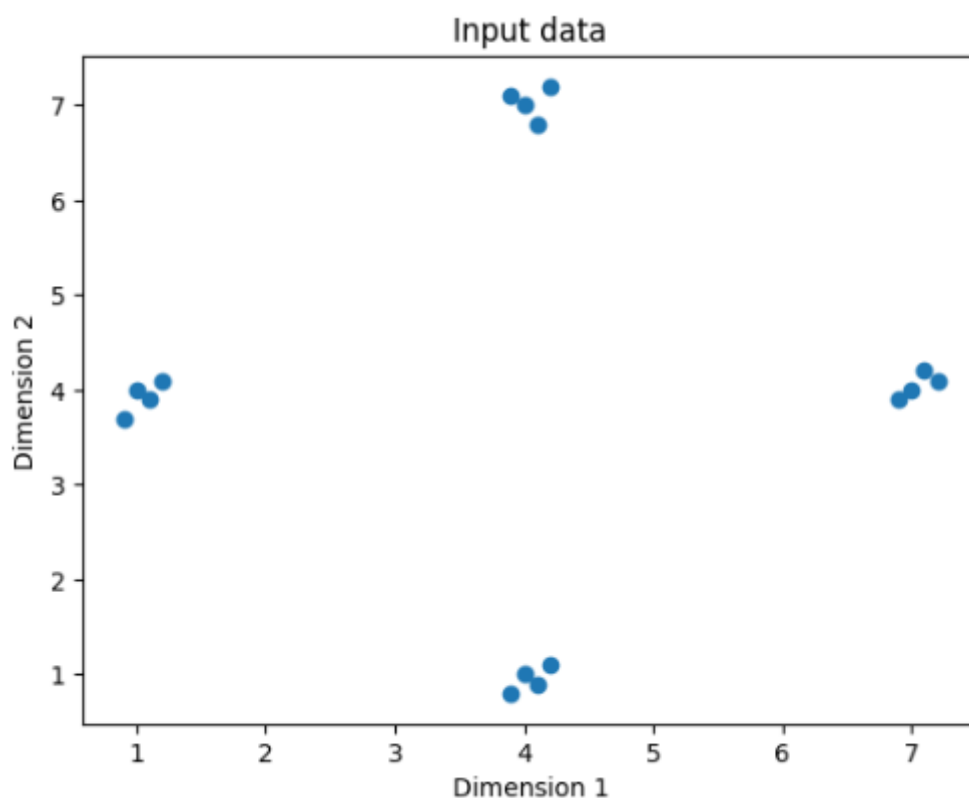
```
# Plot the training progress
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Training error')
plt.title('Training error progress')
plt.grid()

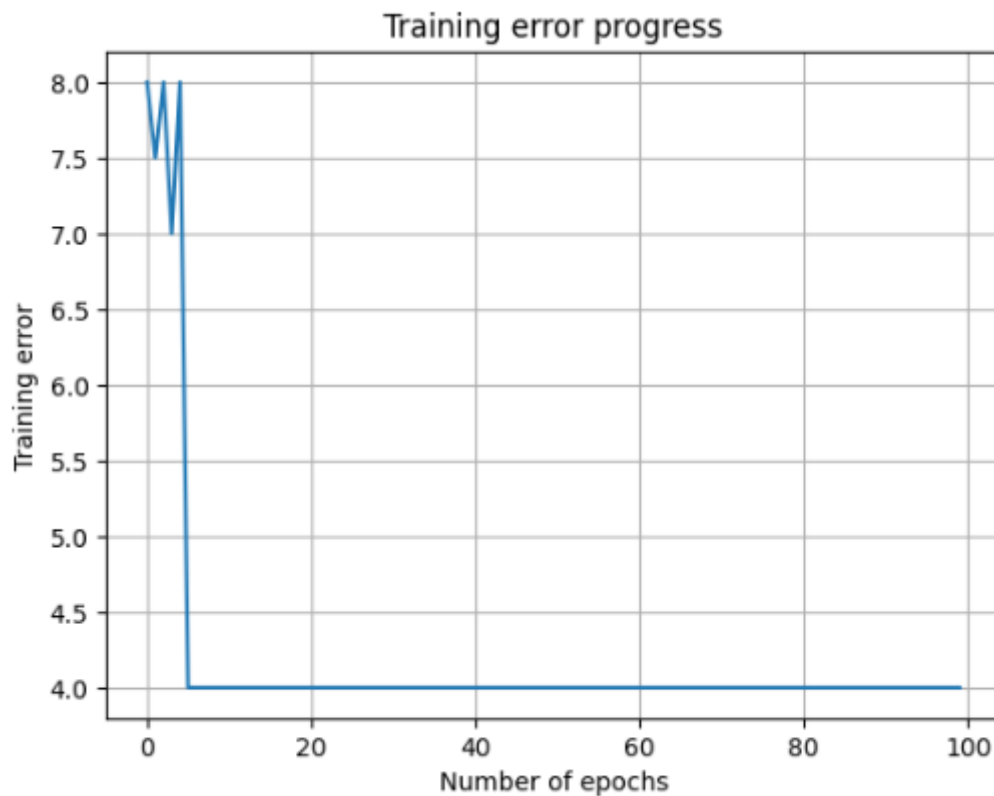
plt.show()
```

10. Define some sample test data points and run the network on those points:

```
print('\nTest results:')
data_test = [[0.4, 4.3], [4.4, 0.6], [4.7, 8.1]]
for item in data_test:
    print(item, '-->', nn.sim([item])[0])
```

11. If you run the program, you will see two figures and test results. The first figure is the input data points. The second one is the training error rate performance.





```
Epoch: 20; Error: 4.0;
Epoch: 40; Error: 4.0;
Epoch: 60; Error: 4.0;
Epoch: 80; Error: 4.0;
Epoch: 100; Error: 4.0;
The maximum number of train epochs is reached

Test results:
[0.4, 4.3] --> [ 0.  0.]
[4.4, 0.6] --> [ 1.  0.]
[4.7, 0.1] --> [ 1.  1.]
```



Workshop 10.2 : Single-layered Neural Networks (SLNN)

1. Combine all the above code and named it as **slnn.py**.
2. If everything is okay, you should see the above 2 figures, training epochs and test results.
3. Save and submit the source code and these figures.
4. As shown in the second figure, why the error rate cannot drop to zero like the simple perceptron?
5. Use different learning rate e.g. 1.0, 0.5, 0.2, 0.0002, etc, to check if there is any difference in terms of system performance. Why?

10.7 Constructing a Multi-layer Neural Network

So, we enhanced our model from a few nodes to a single-layer, but we are still far away from 85 billion nodes.

We won't get to that in this section either, but let's take another step in the right direction.

The human brain does not use a single-layer model.

The output from some neurons becomes the input for other neurons and so on.

A model that has this characteristic is known as a multi-layer neural network.

This type of architecture yields higher accuracy, and it enables us to solve more complex and more varied problems.

Let's see how we can use NeuroLab to build a multi-layer neural network.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

2. In the previous two sections, we saw how to use a neural network as a classifier. In this section, we will see how to use a multi-layer neural network as a regressor. Generate some sample data points based on the equation $y = 3x^2 + 5$ and then normalize the points:

```
# Generate some training data
min_val = -15
max_val = 15
num_points = 130
x = np.linspace(min_val, max_val, num_points)
y = 3 * np.square(x) + 5
y /= np.linalg.norm(y)
```

3. Reshape the preceding variables to create a training dataset:

```
# Create data and labels
data = x.reshape(num_points, 1)
labels = y.reshape(num_points, 1)
```

4. Plot the input data:

```
# Plot input data
plt.figure()
plt.scatter(data, labels)
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
```

```
plt.title('Input data')
```

5. Define a multi-layer neural network with two hidden layers. You are free to design a neural network any way you want. For this case, let's use 10 neurons in the first layer and 6 neurons in the second layer. Our task is to predict the value, so the output layer will contain a single neuron:

```
# Define a multilayer neural network with 2 hidden layers;
# First hidden layer consists of 10 neurons
# Second hidden layer consists of 6 neurons
# Output layer consists of 1 neuron
nn = nl.net.newff([[min_val, max_val]], [10, 6, 1])
```

6. Set the training algorithm to gradient descent:

```
# Set the training algorithm to gradient descent
nn.trainf = nl.train.train_gd
```

7. Train the neural network using the training data that was generated:

```
# Train the neural network
error_progress = nn.train(data, labels, epochs=2000, show=100, goal=0.01)
```

8. Run the neural network on the training data points:

```
# Run the neural network on training datapoints
output = nn.sim(data)
y_pred = output.reshape(num_points)
```

9. Plot the training progress:

```
# Plot training error
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Error')
plt.title('Training error progress')
```

10. Plot the predicted output:

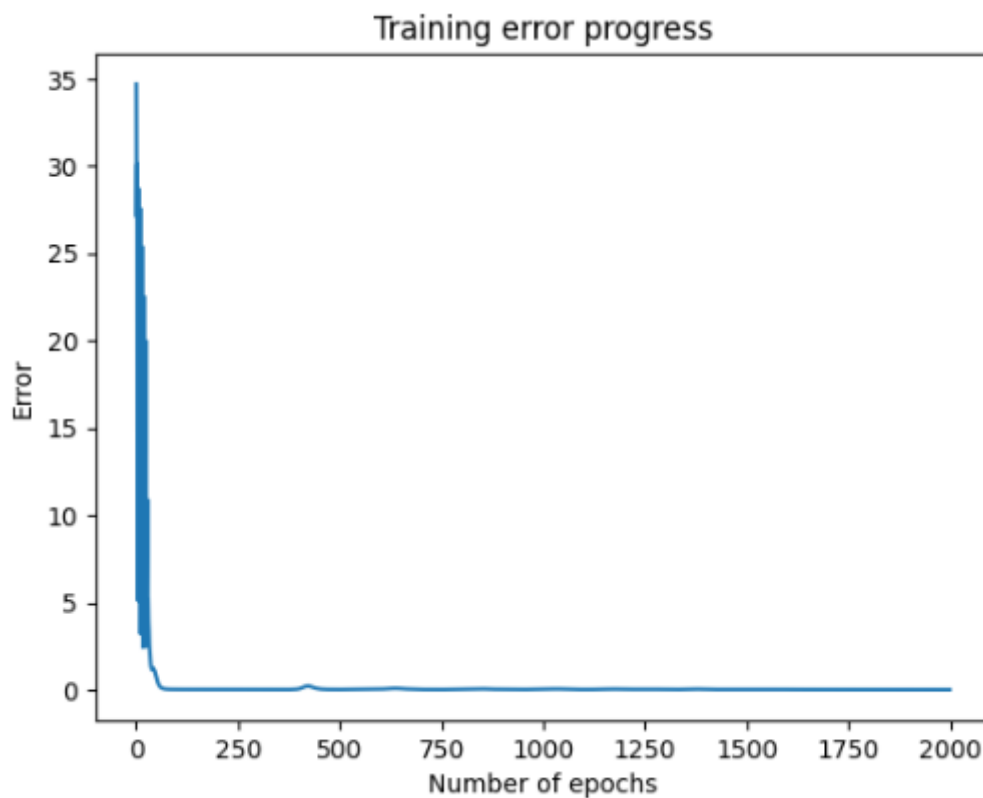
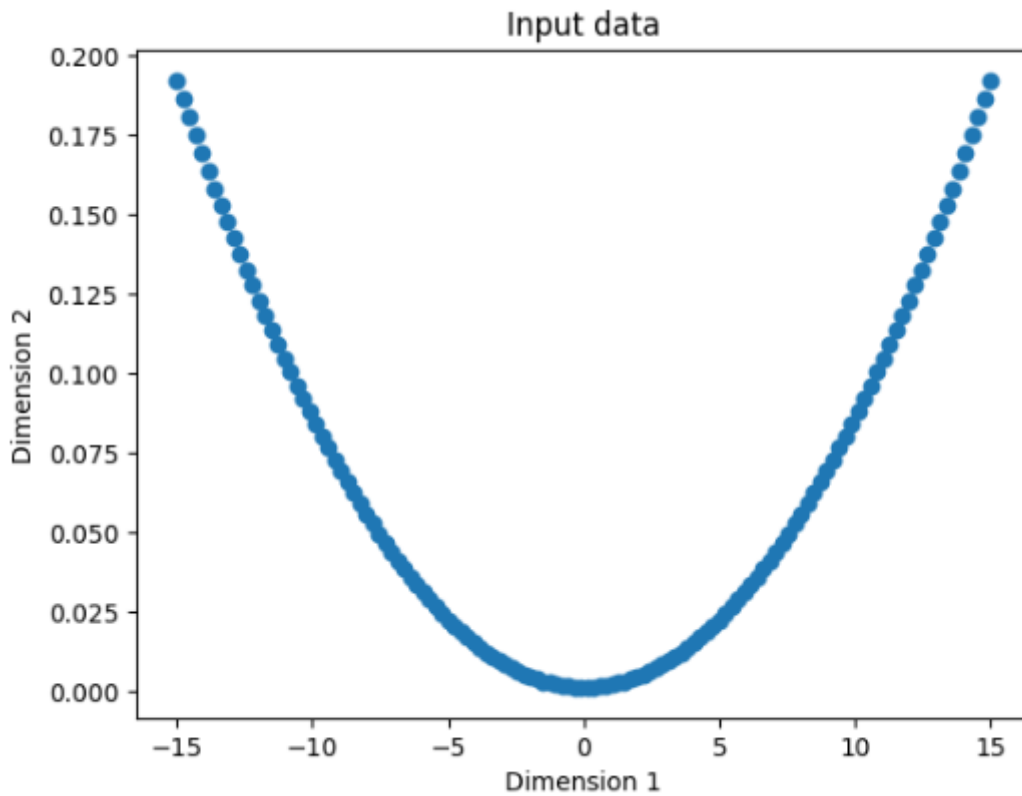
```
# Plot the output
x_dense = np.linspace(min_val, max_val, num_points * 2)
y_dense_pred = nn.sim(x_dense.reshape(x_dense.size,1)).reshape(x_dense.size)

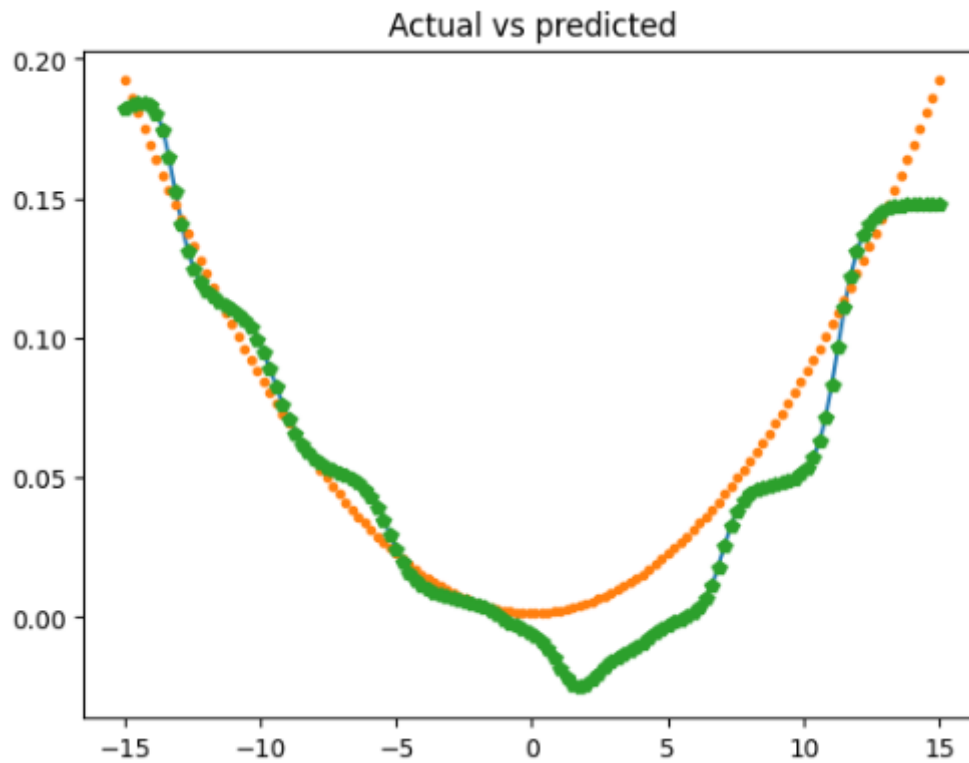
plt.figure()
plt.plot(x_dense, y_dense_pred, '-', x, y, '.', x, y_pred, 'p')
plt.title('Actual vs predicted')
```

```
plt.show()
```

11. If you run the program, you will see the following:

- The first figure shows the input data generated from the equation.
- The second figure shows the training progress
- The third figure shows the predicted output overlaid on the input data.
- The fourth figure shows the training epochs from 100 to 2000.





```
Epoch: 100; Error: 0.035811176231214804;
Epoch: 200; Error: 0.03028737293346494;
Epoch: 300; Error: 0.027085779444603247;
Epoch: 400; Error: 0.06356080433737502;
Epoch: 500; Error: 0.029662346367496616;
Epoch: 600; Error: 0.04715088615886221;
Epoch: 700; Error: 0.032366146682750806;
Epoch: 800; Error: 0.04273976506125249;
Epoch: 900; Error: 0.036512470574726355;
Epoch: 1000; Error: 0.04486505431339596;
Epoch: 1100; Error: 0.030455820824484493;
Epoch: 1200; Error: 0.04720370066505672;
Epoch: 1300; Error: 0.030786247605950844;
Epoch: 1400; Error: 0.04467365719446342;
Epoch: 1500; Error: 0.02968875933162094;
Epoch: 1600; Error: 0.030483352839835012;
Epoch: 1700; Error: 0.024870564807972794;
Epoch: 1800; Error: 0.023409046025639528;
Epoch: 1900; Error: 0.02122834393458827;
Epoch: 2000; Error: 0.019438352696225833;
The maximum number of train epochs is reached
```



Workshop 10.3 : Multi-layered Neural Networks (MLNN)

1. Combine all the above code and named it as `mlnn.py`.

2. If everything is okay, you should see the above figures, training epochs and test results.
3. Save and submit the source code and these figures.
4. By fine tuning the parameters, such as the number of neurons in the first/second layer, learning rate, try to improve the training and testing performance, named your program as **mlnn_2.py**.
5. Add one more hidden layer and do the test again. Save and submit the program as **mlnn_3.py**.
6. Does adding one more hidden layer can improve the performance? Why?

10.8 Building a Vector Quantizer

Vector Quantization is a quantization technique where the input data is represented by a fixed number of representative points. It is the N-dimensional equivalent of rounding off a number.

This technique is commonly used in multiple fields such as **voice/image recognition, semantic analysis, and image/voice compression**.

The history of optimal vector quantization theory goes back to the 1950s in Bell Labs, where research was carried out to optimize signal transmission using discretization procedures.

One advantage of vector quantizer neural networks is that they have high interpretability.

Let's see how we can build a vector c.



Due to some issues with the current version of NeuroLab (v. 0.3.5), running the following code will throw an error. Fortunately, there is a fix for this, but it involves making a change in the NeuroLab package (located at Python38\Lib\site-packages\neurolab). Changing line 179 of the net.py file in the NeuroLab package (layer_out. np['w'][n][st:i].fill(1.0)) to layer_out.np['w'][n][int(st):int(i)].fill(1.0)) should fix this issue..

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

2. Load the input data from the file data_vector_quantization.txt. Each line in this file contains six numbers. The first two numbers form the datapoint and the last four numbers form a one-hot encoded label. There are four classes overall.

```
# Load input data
text = np.loadtxt('data_vector_quantization.txt')
```

3. Separate the text into data and labels:

```
# Separate it into data and labels
data = text[:, 0:2]
labels = text[:, 2:]
```

4. Define a neural network with two layers where we have 10 neurons in the input layer and 4 neurons in the output layer:

```
# Define a neural network with 2 layers:
# 10 neurons in input layer and 4 neurons in output layer
num_input_neurons = 10
num_output_neurons = 4
weights = [1/num_output_neurons] * num_output_neurons
nn = nl.net.newlvq(nl.tool.minmax(data), num_input_neurons, weights)
```

5. Train the neural network using the training data:

```
# Train the neural network
_ = nn.train(data, labels, epochs=500, goal=-1)
```

6. In order to visualize the output clusters, let's create a grid of points:

```
# Create the input grid
xx, yy = np.meshgrid(np.arange(0, 10, 0.2), np.arange(0, 10, 0.2))
xx.shape = xx.size, 1
yy.shape = yy.size, 1
grid_xy = np.concatenate((xx, yy), axis=1)
```

7. Evaluate the grid of points using the neural network:

```
# Evaluate the input grid of points
grid_eval = nn.sim(grid_xy)
```

8. Extract the four classes:

```
# Define the 4 classes
class_1 = data[labels[:,0] == 1]
class_2 = data[labels[:,1] == 1]
class_3 = data[labels[:,2] == 1]
class_4 = data[labels[:,3] == 1]
```

9. Extract the grids corresponding to those four classes:

```
# Define X-Y grids for all the 4 classes
grid_1 = grid_xy[grid_eval[:,0] == 1]
grid_2 = grid_xy[grid_eval[:,1] == 1]
grid_3 = grid_xy[grid_eval[:,2] == 1]
grid_4 = grid_xy[grid_eval[:,3] == 1]
```

10. Plot the outputs:

```
# Plot the outputs
plt.plot(class_1[:,0], class_1[:,1], 'ko',
         class_2[:,0], class_2[:,1], 'ko',
```

```

class_3[:,0], class_3[:,1], 'ko',
class_4[:,0], class_4[:,1], 'ko')
plt.plot(grid_1[:,0], grid_1[:,1], 'm.',
grid_2[:,0], grid_2[:,1], 'bx',
grid_3[:,0], grid_3[:,1], 'c^',
grid_4[:,0], grid_4[:,1], 'y+')
plt.axis([0, 10, 0, 10])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Vector quantization')

plt.show()

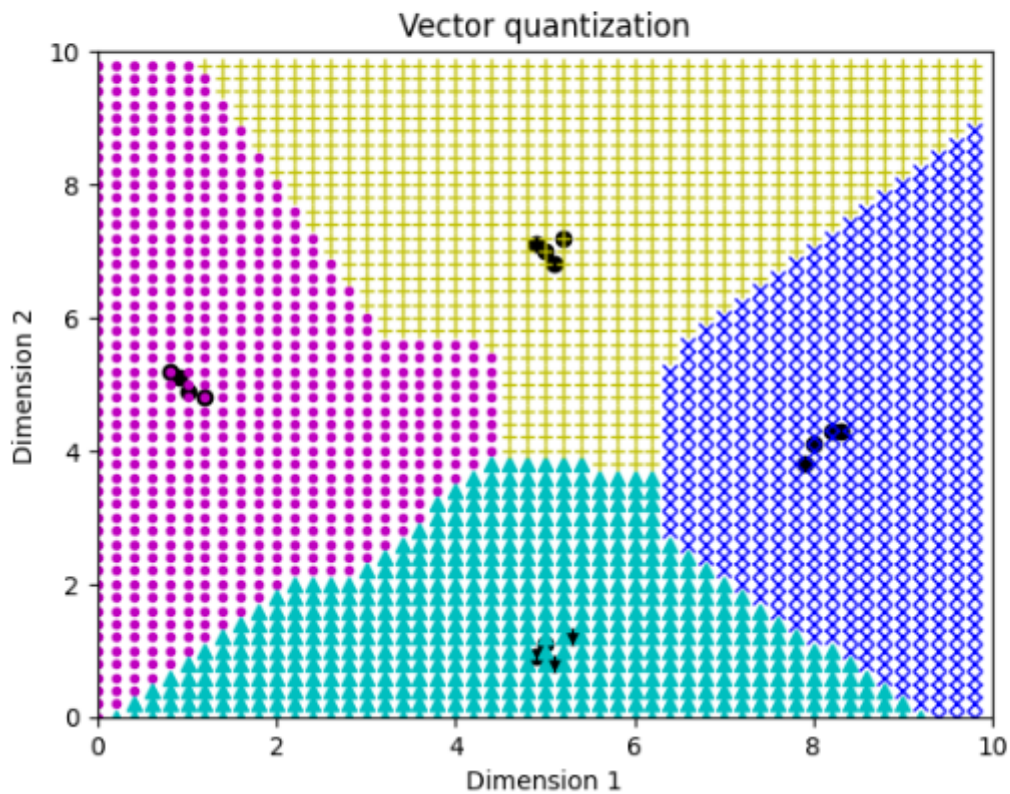
```

11. If you NeuroLab bug (as notice) and run the program, you will see the following Vector quantization plot clusters with the training results.

```

Epoch: 100; Error: 0.0;
Epoch: 200; Error: 0.0;
Epoch: 300; Error: 0.0;
Epoch: 400; Error: 0.0;
Epoch: 500; Error: 0.0;
The maximum number of train epochs is reached

```



Workshop 10.4 : Vector Quantizer

1. Combine all the above code and named it as **VQ.py**.
2. If everything is okay, you should see the above figures, training epoches and VQ clustering result.
3. Save and submit the source code and these figures.
4. By fine tuning the network parameter(s), try to further improve the performance of the VQ system, Save and submit it as **VQ1.py**.
5. Compare the VQ clustering program to the traditional unsupervised learning we have learnt in the previous workshop such as K-means or Gaussian Mixture Models, which one is better? Why?

10.9 Analyzing Time Sequential Data using Recurrent Neural Networks

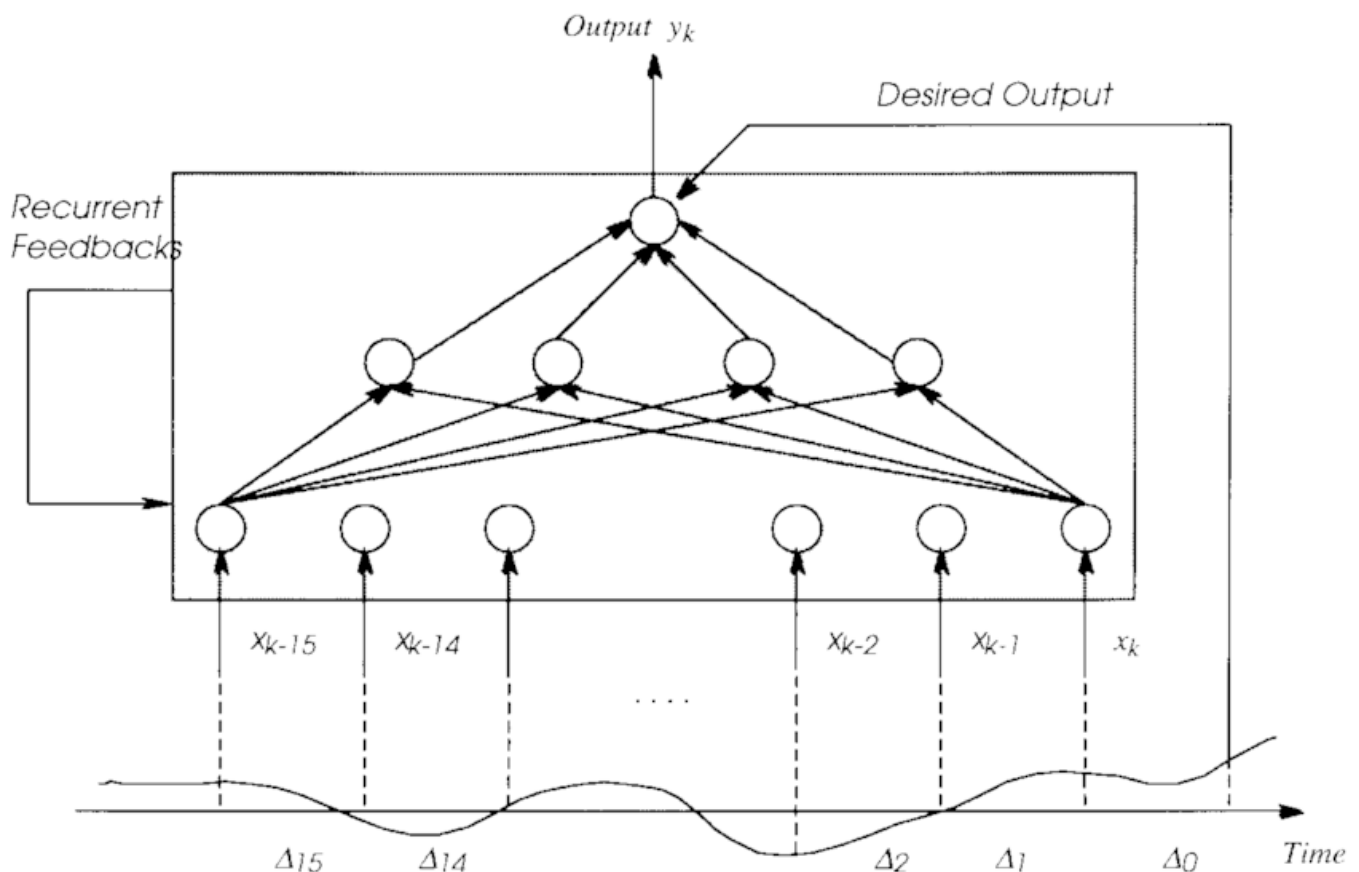
With all our neural network examples so far, we have been using static data.

Neural networks can also be used effectively to build models that process time sequential data.

Recurrent neural networks (RNNs) are great at modeling such kind of sequential data.

When we are working with time series data, we normally cannot use generic learning models.

We need to capture the temporal dependencies in the data so that a robust model can be built.



Let's see how to build it.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import neurolab as nl
```

2. Define a function to generate the waveforms. Start by defining four sine waves:

```
def get_data(num_points):  
    # Create sine waveforms  
    wave_1 = 0.5 * np.sin(np.arange(0, num_points))  
    wave_2 = 3.6 * np.sin(np.arange(0, num_points))  
    wave_3 = 1.1 * np.sin(np.arange(0, num_points))  
    wave_4 = 4.7 * np.sin(np.arange(0, num_points))
```

3. Create varying amplitudes for the overall waveform:

```
# Create varying amplitudes  
amp_1 = np.ones(num_points)  
amp_2 = 2.1 + np.zeros(num_points)  
amp_3 = 3.2 * np.ones(num_points)  
amp_4 = 0.8 + np.zeros(num_points)
```

4. Create the overall waveform:

```
wave = np.array([wave_1, wave_2, wave_3, wave_4]).reshape(num_points * 4, 1)  
amp = np.array([amp_1, amp_2, amp_3, amp_4]).reshape(num_points * 4, 1)  
  
return wave, amp
```

5. Define a function to visualize the output of the neural network:

```
# Visualize the output  
def visualize_output(nn, num_points_test):  
    wave, amp = get_data(num_points_test)  
    output = nn.sim(wave)  
    plt.plot(amp.reshape(num_points_test * 4))  
    plt.plot(output.reshape(num_points_test * 4))
```

6. Define the main function and create a waveform:

```
if __name__ == '__main__':  
    # Create some sample data  
    num_points = 40  
    wave, amp = get_data(num_points)
```

7. Create a recurrent neural network with two layers:

```
# Create a recurrent neural network with 2 layers  
nn = nl.net.newelm([[ -2, 2]], [10, 1], [nl.trans.TanSig(), nl.trans.PureLin()])
```

8. Set the initializer functions for each layer:

```
# Set the init functions for each layer
nn.layers[0].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
nn.layers[1].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
nn.init()
```

9. Train the neural network:

```
# Train the recurrent neural network
error_progress = nn.train(wave, amp, epochs=1200, show=100, goal=0.01)
```

10. Run the data through the network:

```
# Run the training data through the network
output = nn.sim(wave)
```

11. Plot the output:

```
# Plot the results
plt.subplot(211)
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Error (MSE)')

plt.subplot(212)
plt.plot(amp.reshape(num_points * 4))
plt.plot(output.reshape(num_points * 4))
plt.legend(['Original', 'Predicted'])
```

12. Test the performance of the neural network on unknown test data:

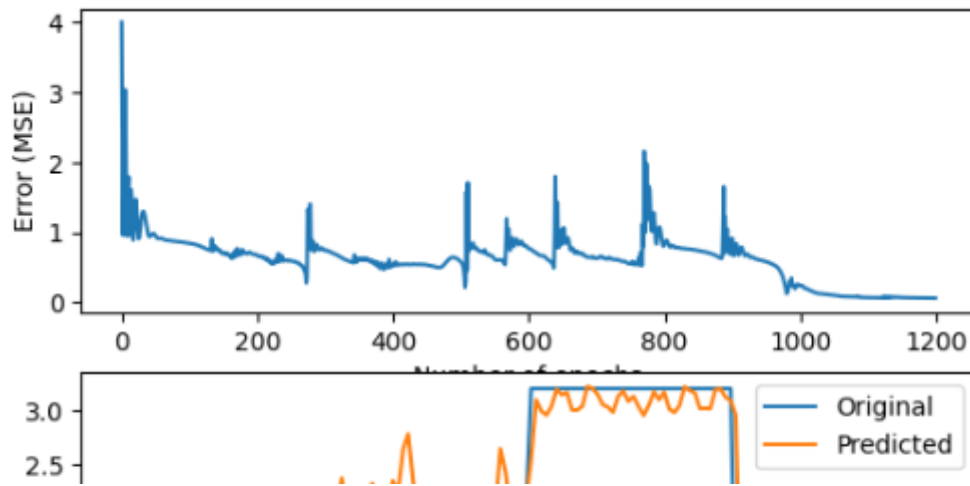
```
# Testing the network performance on unknown data
plt.figure()

plt.subplot(211)
visualize_output(nn, 82)
plt.xlim([0, 300])

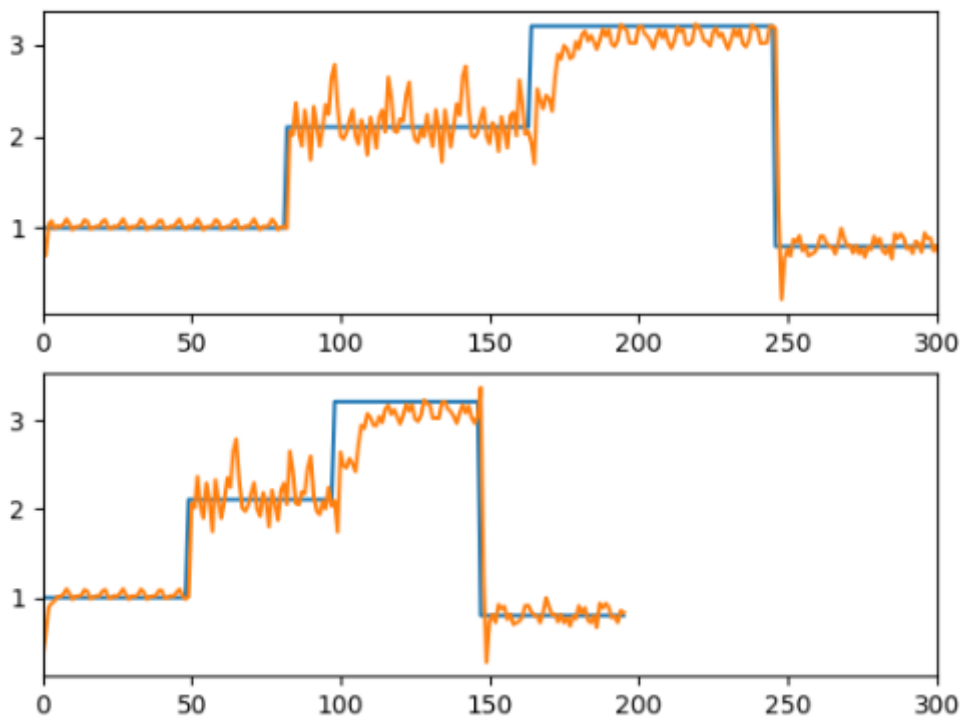
plt.subplot(212)
visualize_output(nn, 49)
plt.xlim([0, 300])

plt.show()
```

13. If you run the program, you will see two output figures. The upper half of the first screenshot shows the training progress and the lower half shows the predicted output overlaid on top of the input waveform:



14. The upper half of the following screenshot shows how the neural network simulates the waveform even though we increased the length of the waveform. The lower half of the screenshot shows the same for decreased length:



15. Here is the training result. What is your findings?

```
Epoch: 100; Error: 0.843603664052111;
Epoch: 200; Error: 0.6750406112428127;
Epoch: 300; Error: 0.7398772222958447;
Epoch: 400; Error: 0.5376201208983902;
Epoch: 500; Error: 0.5865170991104319;
Epoch: 600; Error: 0.7699105434435227;
Epoch: 700; Error: 0.6272063417840193;
Epoch: 800; Error: 0.8062442888811869;
Epoch: 900; Error: 0.8814522691769389;
Epoch: 1000; Error: 0.23628539166532364;
Epoch: 1100; Error: 0.06887653086027863;
Epoch: 1200; Error: 0.0606482805951975;
The maximum number of train epochs is reached
```



Workshop 10.5 : Time Sequential Recurrent Neural Network (TSRNN)

1. Combine all the above code and named it as **tsrnn.py**.
2. If everything is okay, you should see the above figures, training epochs and test result performance.
3. Save and submit the source code and these figures.
4. Does adding more hidden layer can improve the system performance? Add one, two or more layers to check whether it can improve the system performance. Save and submit the program and results. Name the program as **tsrnn_N.py** where N is the number of hidden layers.

10.10 Visualizing characters in an Optical Character Recognition (OCR) database

Neural networks can also be used for **Optical Character Recognition (OCR)**.

It is perhaps one of its most common use cases. Converting handwriting to computer characters has been a fundamental problem that many computer scientists have tried to solve and has yet remained elusive.

We have made great advances but, for obvious reasons, 100% accuracy will remain out of reach. Why?

Think about this scenario: Have you ever written anything down and five minutes later, you are unable to read your own handwriting? Computers will always have this problem as well. There is an infinite number of ways to write down the number 6 and some of them will look more like a 0 or a 5 than a 6. I might be wrong, but I think we will find a cure for cancer before we are able to find a reliable way for a computer to recognize a doctor's handwriting.

We can already achieve high levels of accuracy and the more beautiful the handwriting, the easier it will be to read. The reason we keep on trying to solve this problem is because it's a worthwhile goal that has many applications.

By way of a quick example, a doctor's time is highly valued. As the systems get better at recognizing their notes, they will be freed up to focus their energy on actually treating and helping patients and focus less on paperwork.

Optical Character Recognition (OCR) is the process of recognizing handwritten characters in images. Before building a model, let's familiarize ourselves with the dataset. We will be using the dataset available at:

<http://ai.stanford.edu/~btaskar/ocr> (<http://ai.stanford.edu/~btaskar/ocr>)

You will be downloading a file called letter.data. For convenience, this file has been provided to you in the code bundle.

Let's see how to load that data and visualize the characters.

Before to do so, please install OpenCV package if your system don't have it by using pip command:

pip install opencv-python

1. Create a new Python file and import the following packages:

```
import os
import sys

import cv2
import numpy as np
```

2. Define the input file containing the OCR data:

```
# Define the input file
input_file = 'letter.data'
```

3. Define the visualization and other parameters required to load the data from that file:

```
# Define the visualization parameters
img_resize_factor = 12
start = 6
end = -1
height, width = 16, 8
```

4. Iterate through the lines of that file until the user presses the Esc key. Each line in that file is tab-separated. Read each line and scale it up to 255:

```
# Iterate until the user presses the Esc key
with open(input_file, 'r') as f:
    for line in f.readlines():
        # Read the data
        data = np.array([255 * float(x) for x in line.split('\t')[start:end]])
```

5. Reshape the 1D array into a 2D image:

```
# Reshape the data into a 2D image
img = np.reshape(data, (height, width))
```

6. Scale the image for visualization:

```
# Scale the image
img_scaled = cv2.resize(img, None, fx=img_resize_factor, fy=img_resize_factor)
```

7. Display the image:

```
# Display the image
```

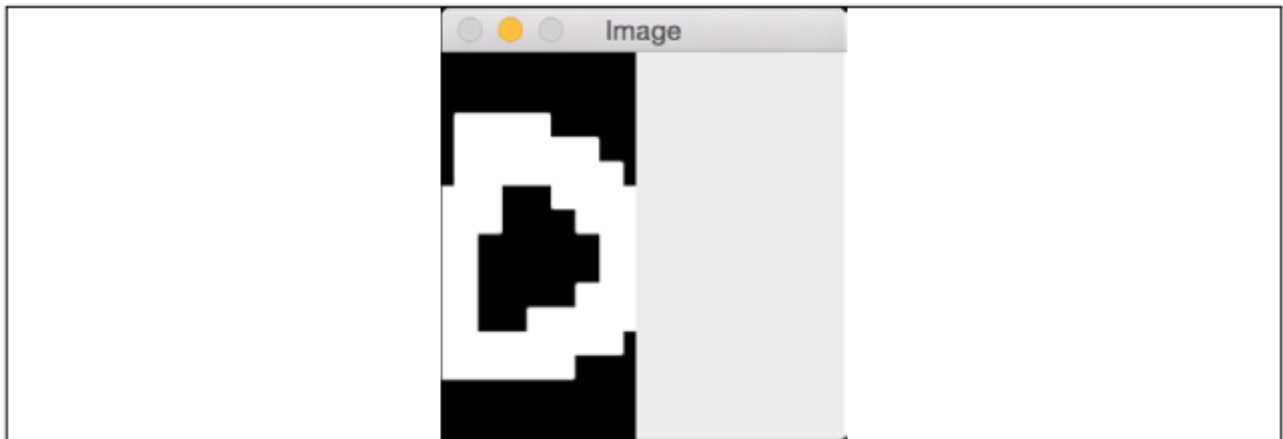
```
cv2.imshow('Image', img_scaled)
```

8. Check if the user has pressed the Esc key. If so, exit the loop:

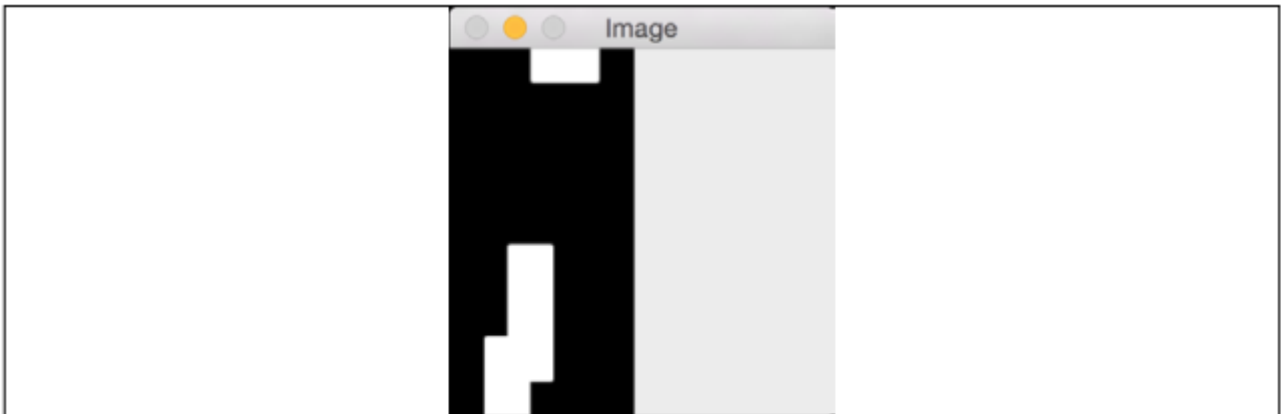
```
# Check if the user pressed the Esc key  
c = cv2.waitKey()  
if c == 27:  
    break
```

9. Run the program, you will get an output screenshot displaying a character. You can keep pressing the space bar to see more characters.

For example, an o might look like this:



And an i might look like this:



Workshop 10.6 : Visualizing Characters in OCR database

1. Combine all the above code and named it as **character_visualizer.py**.
2. If everything is okay, you should see various characters in the OCR database.
3. Study it carefully (will be used in the next OCR program) and capture 5 characters from the system.

10.11 Building an OCR Engine using NeuroLab

Now that we have learned how to work with this data, let's build an optical character recognition system using neural networks.

1. Create a new Python file and import the following packages:

```
import numpy as np
import neurolab as nl
```

2. Define the input file:

```
# Define the input file
input_file = 'letter.data'
```

3. Define the number of data points that will be loaded:

```
# Define the number of datapoints to
# be loaded from the input file
num_datapoints = 50
```

4. Define the string containing all the distinct characters:

```
# String containing all the distinct characters
orig_labels = 'omandig'
```

5. Extract the number of distinct classes:

```
# Compute the number of distinct characters
num_orig_labels = len(orig_labels)
```

6. Define the train and test split. We will use 90% for training and 10% for testing:

```
# Define the training and testing parameters
num_train = int(0.9 * num_datapoints)
num_test = num_datapoints - num_train
```

7. Define the dataset extraction parameters:

```
# Define the dataset extraction parameters
start = 6
```

```
end = -1
```

8. Create the dataset:

```
# Creating the dataset
data = []
labels = []
with open(input_file, 'r') as f:
    for line in f.readlines():
        # Split the current line tabwise
        list_vals = line.split('\t')
```

9. If the label is not in our list of labels, we should skip it:

```
# Check if the label is in our ground truth
# labels. If not, we should skip it.
if list_vals[1] not in orig_labels:
    continue
```

10. Extract the current label and append it to the main list:

```
# Extract the current label and append it
# to the main list
label = np.zeros((num_orig_labels, 1))
label[orig_labels.index(list_vals[1])] = 1
labels.append(label)
```

11. Extract the character vector and append it to the main list:

```
# Extract the character vector and append it to the main list
cur_char = np.array([float(x) for x in list_vals[start:end]])
data.append(cur_char)
```

12. Exit the loop once we have created the dataset:

```
# Exit the loop once the required dataset has been created
if len(data) >= num_datapoints:
    break
```

13. Convert the lists into NumPy arrays:

```
# Convert the data and labels to numpy arrays
data = np.asarray(data)
labels = np.array(labels).reshape(num_datapoints, num_orig_labels)
```

14. Extract the number of dimensions:

```
# Extract the number of dimensions
```

```
num_dims = len(data[0])
```

15. Create a feedforward neural network and set the training algorithm to gradient descent:

```
# Create a feedforward neural network
nn = nl.net.newff([[0, 1] for _ in range(len(data[0]))],
                  [128, 16, num_orig_labels])

# Set the training algorithm to gradient descent
nn.trainf = nl.train.train_gd
```

16. Train the neural network:

```
# Train the network
error_progress = nn.train(data[:num_train,:], labels[:num_train,:],
                          epochs=10000, show=100, goal=0.01)
```

17. Predict the output for test data:

```
# Predict the output for test inputs
print('\nTesting on unknown data:')
predicted_test = nn.sim(data[num_train:, :])
for i in range(num_test):
    print('\nOriginal:', orig_labels[np.argmax(labels[i])])
    print('Predicted:', orig_labels[np.argmax(predicted_test[i])])
```

18. If you run the code, you should see the following output:

```
Epoch: 100; Error: 65.31070719261682;
Epoch: 200; Error: 29.966041708291023;
Epoch: 300; Error: 34.83434746859463;
Epoch: 400; Error: 35.05787347476158;
Epoch: 500; Error: 20.973884879383014;
Epoch: 600; Error: 18.473010550648944;
Epoch: 700; Error: 19.64618343797091;
Epoch: 800; Error: 18.264773989137687;
Epoch: 900; Error: 18.872332019460544;
Epoch: 1000; Error: 19.57843246181062;
Epoch: 1100; Error: 18.741961887539592;
Epoch: 1200; Error: 5.4027807445826275;
Epoch: 1300; Error: 4.076448560266719;
Epoch: 1400; Error: 3.9682432372044416;
Epoch: 1500; Error: 3.893710786123632;
Epoch: 1600; Error: 3.8415351118934944;
Epoch: 1700; Error: 3.80368934568107;
Epoch: 1800; Error: 3.773676012228064;
Epoch: 1900; Error: 3.7480537285563815;
Epoch: 2000; Error: 3.7296980040070813;
Epoch: 2100; Error: 3.749560533015998;
Epoch: 2200; Error: 3.6532500943564523;
Epoch: 2300; Error: 3.5522523024536152;
Epoch: 2400; Error: 3.5349624046557295;
Epoch: 2500; Error: 3.5501783402592517;
Epoch: 2600; Error: 3.5429343932740323;
Epoch: 2700; Error: 3.6491989169272636;
Epoch: 2800; Error: 3.4914468337897704;
Epoch: 2900; Error: 3.493460419301151;
Epoch: 3000; Error: 3.477208767362696;
```

It will keep going until 10,000 epochs. Once it's done, you should see the following output:

```
Epoch: 10000; Error: 3.2417648560235595;  
The maximum number of train epochs is reached
```

```
Testing on unknown data:
```

```
Original: o  
Predicted: o
```

```
Original: m  
Predicted: n
```

```
Original: m  
Predicted: m
```

```
Original: a  
Predicted: n
```

```
Original: n  
Predicted: n
```



Workshop 10.7 : OCR Engine

1. Combine all the above code and named it as **ocr_characters.py**.
2. If everything is okay, you should see the training epochs and the prediction results.
3. What is your findings and comments on the prediction performance?
4. If such system can be used for OCR of English handwritten characters. How about Chinese handwritten characters?
5. Using the method for this program, try a small database of ANY 20 Chinese characters, see whether it works. Name the program as **ocr_Chn_characters.py**.
6. Will the system performance be better if you are use a bigger dataset and train longer? Try and give your observations.

Summary

In this workshop, we learned about neural networks.

We discussed how to build and train neural networks.

We talked about perceptrons and built a classifier based on that.

We learned about single-layer neural networks as well as multi-layer neural networks.

We discussed how neural networks could be used to build a vector quantizer.

We analyzed sequential data using recurrent neural networks.

We then built an optical character recognition engine using neural networks.

In the next workshop, we will explore Deep Learning with Convolutional Neural Networks.

In []:

